

RetailHub — AWS Glue + Athena

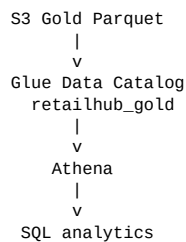
End-to-End Deployment, Failure Analysis, Schema Registration & Query Reference

Purpose: Document the intended Glue + Athena design, the crawler failure, the final Batch EC2 schema-registration solution, the fault-tolerant script, verification commands, and Athena queries.

Region: ap-south-1 **Gold:** s3://retailhub-gold-252bda/ **Glue DB:** retailhub_gold

1. Intent

The intended analytics path is **S3 Gold** → **Glue Data Catalog** → **Athena**. Spark Batch creates Gold Parquet in S3. Glue stores table metadata and S3 locations. Athena runs SQL against the actual Parquet data.



Important: Glue does not store a second copy of the Parquet records. S3 is the data; Glue is the catalog; Athena is the query engine.

2. Intended Gold Tables

- fact_orders
- fact_order_items
- fact_payments
- fact_delivery
- fact_product_interactions
- dim_customer
- dim_product
- dim_category
- dim_date

Each table maps to s3://retailhub-gold-252bda/<table_name>/. The internal **_state/** area was not intended as a business table.

3. First Approach — Glue Crawler

We attempted one crawler named **retailhub-gold-crawler** with nine individual S3 targets so Glue could infer Parquet schemas automatically.

```
s3://retailhub-gold-252bda/dim_customer/  
s3://retailhub-gold-252bda/fact_orders/  
s3://retailhub-gold-252bda/fact_order_items/  
s3://retailhub-gold-252bda/fact_payments/  
s3://retailhub-gold-252bda/fact_delivery/  
s3://retailhub-gold-252bda/fact_product_interactions/  
s3://retailhub-gold-252bda/dim_product/  
s3://retailhub-gold-252bda/dim_category/  
s3://retailhub-gold-252bda/dim_date/
```

4. What Failed

CloudTrail recorded the exact failure:

```
eventSource: glue.amazonaws.com
eventName: CreateCrawler
awsRegion: ap-south-1
userIdentity: Root
errorCode: AccessDenied
errorMessage:
Account 939490550970 is denied access.
```

The CloudTrail request also showed the nine S3 targets were actually submitted. The Glue role trust relationship allowed **glue.amazonaws.com** to assume the role with the source account restriction. Because the caller was Root and the denial was account-level on CreateCrawler, we stopped changing IAM blindly and used direct catalog registration instead.

5. Final Solution

Batch EC2 already had Spark, S3 access and the RetailHub Batch IAM role. We used it to read each real Gold Parquet schema and register/update only the metadata in Glue.

```
Existing Gold Parquet
|
| Spark reads
v
Actual schema
|
| Python converts
v
Glue TableInput
|
| create_table / update_table
v
Glue Data Catalog
```

No Parquet records are copied into Glue.

6. Batch IAM Permission

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "RetailHubGlueCatalogAccess",
    "Effect": "Allow",
    "Action": [
      "glue:GetDatabase",
      "glue:GetDatabases",
      "glue:GetTable",
      "glue:GetTables",
      "glue:CreateTable",
      "glue:UpdateTable"
    ],
    "Resource": "*"
  }]
}
```

7. Full Schema Registration Script

This is the final fault-tolerant version used on Batch EC2.

```
import boto3

from pyspark.sql import SparkSession
from pyspark.sql.utils import AnalysisException

AWS_REGION = "ap-south-1"
DATABASE = "retailhub_gold"
GOLD_BUCKET = "retailhub-gold-252bda"

TABLES = [
    "fact_orders",
    "fact_order_items",
    "fact_payments",
    "fact_delivery",
    "fact_product_interactions",
    "dim_customer",
    "dim_product",
    "dim_category",
    "dim_date",
]

spark = (
    SparkSession.builder
    .appName("RetailHub-Create-Glue-Tables")
    .getOrCreate()
)

glue = boto3.client("glue", region_name=AWS_REGION)
```

```

s3 = boto3.client("s3", region_name=AWS_REGION)

parquet_input_format = (
    "org.apache.hadoop.hive ql.io.parquet.MapredParquetInputFormat"
)
parquet_output_format = (
    "org.apache.hadoop.hive ql.io.parquet.MapredParquetOutputFormat"
)
parquet_serde = (
    "org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe"
)

created_tables = []
updated_tables = []
skipped_tables = []
failed_tables = []

def spark_type_to_glue_type(data_type):
    dt = data_type.simpleString()

    mapping = {
        "string": "string",
        "boolean": "boolean",
        "byte": "tinyint",
        "short": "smallint",
        "int": "int",
        "long": "bigint",
        "float": "float",
        "double": "double",
        "date": "date",
        "timestamp": "timestamp",
        "binary": "binary",
    }

    if dt in mapping:
        return mapping[dt]

    if dt.startswith("decimal"):
        return dt
    if dt.startswith("array<"):
        return dt
    if dt.startswith("map<"):
        return dt
    if dt.startswith("struct<"):
        return dt

    return "string"

def s3_path_exists(bucket, prefix):
    try:
        response = s3.list_objects_v2(
            Bucket=bucket,
            Prefix=prefix,
            MaxKeys=1
        )
        return len(response.get("Contents", [])) > 0

    except Exception as e:
        print(
            f"WARNING: Could not check "
            f"s3://{bucket}/{prefix}"
        )
        print(f"S3 error: {e}")
        return False

for table_name in TABLES:

    print()
    print("=" * 90)
    print(f"PROCESSING TABLE: {table_name}")
    print("=" * 90)

    s3_prefix = f"{table_name}/"
    s3_path = f"s3a://{GOLD_BUCKET}/{s3_prefix}"
    glue_location = f"s3://{GOLD_BUCKET}/{s3_prefix}"

    print(f"S3 Path : {s3_path}")
    print(f"Glue Table : {table_name}")

    if not s3_path_exists(GOLD_BUCKET, s3_prefix):

        print("SKIPPED: S3 path does not exist:")
        print(f" {glue_location}")

        skipped_tables.append(table_name)
        continue

    try:
        df = spark.read.parquet(s3_path)

        print()

```

```

        print("Spark schema:")
        df.printSchema()

    except AnalysisException as e:

        print(f"SKIPPED: Spark could not read {table_name}")
        print(f"Reason: {e}")

        skipped_tables.append(table_name)
        continue

    except Exception as e:

        print(f"FAILED while reading {table_name}")
        print(f"Error: {e}")

        failed_tables.append(table_name)
        continue

    try:

        columns = []

        for field in df.schema.fields:

            glue_type = spark_type_to_glue_type(
                field.dataType
            )

            columns.append({
                "Name": field.name,
                "Type": glue_type
            })

        print()
        print("Glue columns:")

        for column in columns:
            print(
                f"    {column['Name']} : "
                f"{column['Type']}"
            )

    except Exception as e:

        print(
            f"FAILED while building schema "
            f"for {table_name}"
        )
        print(f"Error: {e}")

        failed_tables.append(table_name)
        continue

    table_input = {

        "Name": table_name,

        "Description": (
            f"RetailHub Gold table generated "
            f"from {table_name} Parquet data"
        ),

        "TableType": "EXTERNAL_TABLE",

        "Parameters": {
            "classification": "parquet",
            "EXTERNAL": "TRUE"
        },

        "StorageDescriptor": {

            "Columns": columns,

            "Location": glue_location,

            "InputFormat": parquet_input_format,

            "OutputFormat": parquet_output_format,

            "SerdeInfo": {
                "SerializationLibrary": parquet_serde,
                "Parameters": {
                    "serialization.format": "1"
                }
            },

            "Compressed": False,
            "StoredAsSubDirectories": False
        }
    }

    try:

```

```

try:
    glue.get_table(
        DatabaseName=DATABASE,
        Name=table_name
    )
    table_exists = True

except glue.exceptions.EntityNotFoundException:
    table_exists = False

if not table_exists:

    glue.create_table(
        DatabaseName=DATABASE,
        TableInput=table_input
    )

    print(
        f"SUCCESS: Created Glue table "
        f"{table_name}"
    )

    created_tables.append(table_name)

else:

    glue.update_table(
        DatabaseName=DATABASE,
        TableInput=table_input
    )

    print(
        f"SUCCESS: Updated Glue metadata "
        f"for {table_name}"
    )

    updated_tables.append(table_name)

except Exception as e:

    print(
        f"FAILED creating/updating "
        f"{table_name}"
    )
    print(f"Error: {e}")

    failed_tables.append(table_name)
    continue

print()
print("=" * 90)
print("RETAILHUB GLUE CATALOG SUMMARY")
print("=" * 90)

print()
print("CREATED TABLES:")
if created_tables:
    for table in created_tables:
        print(f" + {table}")
else:
    print(" None")

print()
print("UPDATED TABLES:")
if updated_tables:
    for table in updated_tables:
        print(f" ~ {table}")
else:
    print(" None")

print()
print("SKIPPED TABLES:")
if skipped_tables:
    for table in skipped_tables:
        print(f" - {table}")
else:
    print(" None")

print()
print("FAILED TABLES:")
if failed_tables:
    for table in failed_tables:
        print(f" ! {table}")
else:
    print(" None")

print()
print("=" * 90)
print("GLUE CATALOG PROCESSING COMPLETE")
print("=" * 90)

spark.stop()

```

8. How the Script Gets the Schema

The script reads the real Parquet data location with Spark:

```
s3_path = f"s3a://{GOLD_BUCKET}/{s3_prefix}"
df = spark.read.parquet(s3_path)
```

It then extracts the real schema and converts Spark types into Glue types:

```
for field in df.schema.fields:
    glue_type = spark_type_to_glue_type(field.dataType)

    columns.append({
        "Name": field.name,
        "Type": glue_type
    })
```

The Glue metadata points to the existing S3 location:

```
table_input = {
    "Name": table_name,
    "TableType": "EXTERNAL_TABLE",
    "StorageDescriptor": {
        "Columns": columns,
        "Location": glue_location,
        ...
    }
}
```

Then the AWS API registers metadata only:

```
glue.create_table(
    DatabaseName=DATABASE,
    TableInput=table_input
)

# existing table:
glue.update_table(
    DatabaseName=DATABASE,
    TableInput=table_input
)
```

9. Exception Handling — Why It Was Added

The first script stopped at **fact_product_interactions** because the S3 prefix did not exist. The observed error was:

```
[PATH_NOT_FOUND] Path does not exist:
s3a://retailhub-gold-252bda/fact_product_interactions
SQLSTATE: 42K03
```

The final version prevents one missing or bad table from stopping all other tables.

- boto3 checks whether each S3 prefix has objects before Spark reads it.
- Spark AnalysisException is caught and the table is marked SKIPPED.
- Schema-generation errors are caught per table.
- Glue create/update errors are caught per table.
- The loop continues and a final CREATED / UPDATED / SKIPPED / FAILED summary is printed.

10. Existing Tables = Metadata

For a registered table such as **fact_orders**, Glue stores the table name, database, column definitions and an S3 StorageDescriptor location such as:

```
Name:
    fact_orders

Database:
    retailhub_gold

Columns:
    order_id      string
    customer_id   string
    anonymous_id   string
    order_date     date
    order_created_at timestamp
    order_confirmed_at timestamp
    ...
```

Location:
s3://retailhub-gold-252bda/fact_orders/

The records remain in S3. Glue does not duplicate them.

11. Verification Commands

List catalog tables:

```
aws glue get-tables --region ap-south-1 --database-name retailhub_gold --query 'TableList[].Name' --output table
```

Inspect a table:

```
aws glue get-table --region ap-south-1 --database-name retailhub_gold --name fact_orders
```

Run the registration utility from Batch EC2:

```
docker run --rm --env-file /opt/retailhub-batch/.env -v /opt/retailhub-batch/create_glue_tables.py:/opt/retailhub/create_glue_tables.py
```

12. Athena — Purpose

Athena is the serverless SQL engine. It uses Glue metadata to resolve the table definition and S3 location, then reads the actual Parquet files from S3.

```
Analyst
|
v
Athena
|
| metadata
v
Glue Data Catalog
|
| schema + S3 location
v
S3 Gold Parquet
|
v
Results
```

The **retailhub_gold** database is the Glue catalog namespace used by Athena; it is not a separate physical data store.

13. Athena Setup

Open **AWS Console** → **Athena** → **Query editor**. If Athena asks for a query-result location, use a dedicated S3 prefix such as:

```
s3://retailhub-gold-252bda/athena-results/
```

Select the **retailhub_gold** database.

14. Athena Smoke Tests

List tables:

```
SHOW TABLES;
```

Read real Gold records:

```
SELECT *
FROM retailhub_gold.fact_orders
LIMIT 10;
```

Count records:

```
SELECT COUNT(*) AS total_orders
FROM retailhub_gold.fact_orders;
```

These tests validate S3 → Glue → Athena end to end.

15. End-to-End RetailHub Flow

```
RetailHub Website
|
v
Kafka
|
```



16. Why This Design

- Spark remains the compute engine producing Gold Parquet.
- S3 remains the durable Gold data lake.
- Glue provides centralized table metadata.
- Athena provides serverless SQL over S3.
- Batch EC2 reuses existing Spark and IAM capabilities.
- A crawler is optional, not required for the final working design.
- No data is duplicated between S3 and Glue.

17. Failure → Resolution

Problem	Resolution
Glue CreateCrawler returned Account Access Denied	CloudTrail confirmed Root caller and account-level denial; stopped blind IAM changes.
Nine schemas would be tedious to enter manually	Spark inferred schemas from actual Gold Parquet files.
Initial script stopped at missing fact_product_interactions table	Added S3 existence checks and per-table exception handling.
Some Glue tables already existed	Used update_table() for existing metadata.
Whole-bucket crawling could encounter _state	Registered only intended Gold prefixes.

18. Final State

S3 Gold:
retailhub-gold-252bda

Glue database:
retailhub_gold

Glue:
Gold table metadata

Athena:
SQL query engine

Actual data:
remains in S3 as Parquet

19. Re-run Behavior

The utility can be rerun after Gold schemas change. Existing tables are updated from the current Parquet schema; missing prefixes are skipped; unrelated table failures do not stop the run.

20. Terraform Direction

After manual validation, Terraform can represent the stable infrastructure: S3 buckets, Glue database, IAM roles/policies and Athena configuration. The schema-registration utility can remain an explicit deployment/catalog step where appropriate.

```
Terraform
|
+-- S3 buckets
+-- Glue database
+-- IAM roles/policies
+-- Athena configuration
|
v
Catalog registration utility
|
+-- read Gold Parquet schema
+-- create/update Glue tables
|
v
Athena
```

21. Quick Reference

```
# Glue tables
aws glue get-tables --region ap-south-1 --database-name retailhub_gold --query 'TableList[].Name' --output table

# One table
aws glue get-table --region ap-south-1 --database-name retailhub_gold --name fact_orders

# Athena
SHOW TABLES;

SELECT *
FROM retailhub_gold.fact_orders
LIMIT 10;

SELECT COUNT(*) AS total_orders
FROM retailhub_gold.fact_orders;
```

22. Final Mental Model

S3 is the data. Glue is the catalog. Athena is the query engine. Spark creates the Gold Parquet data and can read its real schema to register/update Glue metadata. The crawler was an optional discovery mechanism, not a required part of the final architecture.

End of reference.